ARTEFACT

Take-Home Challenge

Customer NPS Prediction for a Telecom Operator

1. Context

You are joining a fictional data science team inside a pan-African telecom operator. The marketing and customer experience teams run regular Net Promoter Score (NPS) surveys to track customer loyalty, but only a small fraction of the customer base actually responds. The business cannot target retention actions effectively when it only knows the NPS of 15% of its customers.

Your mission: build a machine learning system that predicts a customer's NPS category from their account, service, and interaction data. The predictions will feed a retention workflow that prioritises detractors for proactive outreach.

This is intentionally a realistic exercise rather than a toy modelling task. We want to see how you frame the prediction problem, deal with a noisy and imbalanced target, make reasonable modelling choices, and turn your work into something a business team could actually act on.

Time box

Please plan for a maximum of two weeks. We value thoughtful scope management more than over-engineering.

2. Business objective

Build a machine learning system that predicts the NPS category of a customer (Detractor / Passive / Promoter) from their account and behavioural data, so that the retention team can:

- prioritise outreach towards predicted Detractors before they churn,
- identify the main drivers of detraction, at both individual and segment level,
- simulate the expected NPS of the silent 85% of the base who never answer the survey.

Primary goal: produce a reliable prediction of NPS category at customer level.

Secondary goal: demonstrate sound product, modelling and engineering judgement not only raw performance.

Important nuance: NPS is fundamentally an ordinal target (0 to 10, grouped into three ordered classes). Think carefully about how you formulate the learning problem classification, ordinal classification, regression with thresholding and justify your choice. There is no single right answer; we want to see your reasoning.

3. Required data source

ARTEFACT

Use the IBM Telco Customer Churn dataset, version 11.1.3+, as the core source for this challenge. It is a public, well-documented dataset covering ~7,000 customers of a fictional telecom in California.

Download location (public):

- https://www.kaggle.com/datasets/blastchar/telco-customer-churn
- Alternative: IBM Cognos samples search for 'Telco customer churn (11.1.3+)'

This dataset includes a Satisfaction Score (1 to 5) collected from real customers, which you will use as the basis for building your NPS target variable. The mapping from Satisfaction Score to NPS is part of the exercise see Section 4.1.

You may add external public enrichment data if you believe it improves the model (e.g., U.S. Census-derived features for the ZIP codes present in the dataset, public telecom benchmarks). If you do, explicitly justify the business relevance of each source.

Honesty about the dataset

The IBM Telco dataset is fictional but based on realistic patterns. We deliberately do not ask you to scrape data or generate a fully synthetic one we want you to focus on modelling and product thinking. The satisfaction score is a real human-provided signal inside the dataset, not a fabricated label.

4. Scope of work

Your submission should cover the following areas.

4.1 Building the NPS target

The dataset provides a Satisfaction Score from 1 to 5. You need to derive an NPS label from it and clearly justify your mapping. A reasonable baseline mapping could be:

- Satisfaction 5 → Promoter (NPS 9 - 10)
- Satisfaction 4 → Passive (NPS 7 - 8)
- Satisfaction ≤ 3 → Detractor (NPS 0 - 6)

But this is a starting point, not a recipe. You are encouraged to refine, challenge, or enrich this mapping for example by using Churn Value, CLTV, or tenure as additional signal, or by adding realistic noise. Document your decision and discuss how sensitive your downstream model is to it.

You must also discuss data leakage risks in the target construction itself. In particular: the dataset also contains a Churn Score and a Churn Value. Used naively, these will leak into the target and inflate your metrics. Explain how you handle them.

4.2 Dataset preparation

Create a clean analytical dataset from the IBM data:

ARTEFACT

- Define and document your schema (what you keep, what you drop, what you transform).
- Handle missing, inconsistent, or ambiguous fields transparently.
- Split the data in a way that reflects the real business problem (the 15% of respondents vs the 85% silent base). Explain your validation strategy.
- Discuss class imbalance explicitly NPS class distributions are always skewed. Show you know why this matters.

4.3 Feature engineering

Build and justify relevant features. Examples of signal already present or derivable from the dataset:

- tenure, contract type, payment method
- monthly charges, total charges, and derived ratios (e.g., charges per service)
- service bundle (phone, internet, streaming, security add-ons)
- number of dependents and household composition proxies
- geographic features from ZIP / Latitude / Longitude
- engagement proxies (paperless billing, auto-pay)

You are not required to use every feature. Choose a scope you can execute well. We care about the quality of the features you build, the leakage you avoid, and the clarity with which you explain why each feature should help predict NPS.

4.4 Synthetic customer verbatims (bonus track, optional but encouraged)

In addition to the tabular data, generate one synthetic 'last interaction note' per customer a short free-text message of 1 - 3 sentences simulating what the customer said or wrote during their last contact with customer support (call summary, chat transcript, or app review).

Requirements for this synthetic text:

- Produced using an LLM of your choice, conditioned on a subset of the customer's tabular features (e.g., tenure, contract, services, churn risk).
- Realistic in tone a Detractor should sound frustrated, a Promoter enthusiastic, a Passive neutral. But the correlation should be imperfect (introduce noise and counter-intuitive cases).
- Reproducible fix your seed, store the prompt, and commit the generated text alongside the dataset.

Once generated, you are encouraged to combine this text signal with your tabular features in your model (text embeddings, sentiment features, LLM zero-shot classification, or a hybrid architecture). Explain what the text adds or doesn't add on top of the tabular baseline.

Why this matters

ARTEFACT

In a real telco, verbatims from call centres, chat, and reviews are the richest source of NPS signal but are rarely used well. We want to see if you can combine structured and unstructured data in a business-useful way and if you can spot when the added complexity is not worth it.

4.5 Modelling and evaluation

Train and evaluate one or more models for NPS prediction. We expect you to:

- Start from a sensible baseline (logistic regression, ordinal logistic regression, or a simple gradient boosting model with default parameters). Do not skip this step it disciplines the rest of your work.
- Compare at least two modelling families, justified by the problem. Typical candidates: gradient boosting (XGBoost / LightGBM / CatBoost), ordinal regression, calibrated tree ensembles, or a tabular foundation model (see bonus below).
- Choose evaluation metrics that fit a 3-class, ordered, imbalanced target. Accuracy alone is not enough. Think macro-F1, balanced accuracy, quadratic weighted kappa, per-class recall, calibration plots, lift curves on the Detractor class.
- Define a validation strategy consistent with the business problem the model will be applied to customers who did not answer the survey, so your validation set should simulate that gap.
- Discuss interpretability. At minimum, produce feature importance or SHAP values. Tell us what the model says about drivers of detraction.
- Explicitly discuss leakage, bias, noise, data quality, and the limitations of the label you constructed in 4.1.

(OPTIONAL) Foundation models for tabular data

A bonus point will be given to candidates who successfully explore a tabular foundation model (e.g., TabPFN-2.5, TabICL) on this problem, describing the advantages, limits, and cost/latency trade-offs vs gradient boosting. Keep it honest do not frame it as the winner if it is not.

4.6 Drivers of detraction (bonus, encouraged)

Beyond pure prediction, describe what your model tells you about why customers become detractors. Go beyond a global SHAP summary:

- Segment-level drivers (new customers vs long tenured, month-to-month vs annual contract, fibre vs DSL).
- Actionable vs non-actionable drivers the business can change a contract, it cannot change a demographic.
- A simple recommendation: given a predicted Detractor, what is the single most likely lever?

Causal rigour is not expected but honesty about the difference between correlation and causation is.

4.7 Fairness and bias

The IBM Telco dataset contains demographic attributes (gender, senior citizen flag, partner, dependents) and geographic attributes that can act as proxies for protected characteristics. A model that prioritises retention budget based on predicted NPS can quietly allocate that budget unevenly across groups.

You are expected to:

- Audit your model for disparities in predictive performance across demographic segments at minimum, compare per-group recall for the Detractor class. A model that catches 80% of young detractors but only 50% of senior detractors is not acceptable, even if overall accuracy is high.
- Check whether your features include, or proxy, attributes you would not want to decide on (e.g., ZIP code as a proxy for socio-economic status). Explain what you keep, what you drop, and why.
- Discuss the business consequence of your fairness choices. Removing a feature often costs accuracy. Make the trade-off explicit and defend your position.
- Flag any finding that would need to be escalated to a Customer Experience or Legal team before the model went to production.

We are not looking for a full fairness engineering pipeline. We are looking for a data scientist who does not ship a model blind to its distributional impact, and who can have this conversation with a non-technical executive.

4.8 Model persistence and simple productization

Persist the trained model so it can be reused without retraining.

Build a simple user interface (Streamlit, Gradio, or equivalent) that allows a user a retention manager in the field to:

- input customer attributes (or pick an existing customer ID from the dataset) and obtain a predicted NPS category with probabilities,
- see the top drivers for that specific prediction,
- (bonus) paste a customer verbatim and see how it shifts the prediction.

The interface does not need to look polished. It needs to be usable, clear, and to behave gracefully when some inputs are missing or unknown. Explain the design choices you make.

4.9 Optional extension: monitoring and retraining

Only attempt this section if the rest of your submission is already solid.

- Propose or implement a lightweight monitoring layer that tracks input drift, prediction drift, and performance on newly arriving survey responses.

ARTEFACT

- Propose or implement a retraining trigger (schedule, drift threshold, or a minimum number of new labels).
- Discuss how you would handle the feedback loop between the retention team's actions and the model's future training data.

5. Recommended but optional tools

You may use any stack you prefer. Examples of reasonable choices include:

- Python, Jupyter, pandas, polars, NumPy
- scikit-learn, XGBoost, LightGBM, CatBoost
- mord, statsmodels for ordinal regression
- TabPFN, TabICL, or any tabular foundation model
- sentence-transformers, OpenAI / Anthropic / Mistral API, or local LLMs for verbatim generation and embedding
- SHAP, LIME for interpretation
- MLflow, DVC, Weights & Biases for experiment tracking
- Streamlit, Gradio, FastAPI, Flask, or equivalent

You are not required to use any specific framework. We care more about your choices than the exact tooling.

6. Deliverables

- Source code
- Notebooks, if used
- A README with setup and run instructions, plus a .env.example if relevant
- The derived NPS dataset (or script that produces it deterministically)
- Generated customer verbatims (if you did the bonus), with the generation script
- A trained model artifact
- Interface screenshots and/or a short demo video
- A short write-up (3–6 pages) describing your approach, target construction, modelling decisions, evaluation, drivers of detraction, limitations, and next steps

7. Submission guidance

Make your work reproducible. Someone else should be able to re-run your pipeline from raw data to prediction.

Be explicit about what is implemented, what is approximate, and what is left as future work. We value honesty more than polish.

If you use LLMs or AI tools (including for generating verbatims, scaffolding code, drafting documentation), that is allowed and expected but state clearly what you used them for. You remain responsible for the final quality and correctness.

Please keep credentials and API keys out of the repository.

8. Deadline

Please submit your work within **14 Days** of receiving the challenge.

9. Evaluation philosophy

We are not evaluating raw predictive performance in isolation. We are looking for evidence that you can define and execute a useful ML problem under imperfect, business-realistic conditions.

Concretely, a strong submission demonstrates:

- A correct framing of NPS as an ordered, imbalanced target.
- Honesty about label construction, leakage, and noise.
- A disciplined baseline before any advanced modelling.
- Evaluation metrics aligned with the business problem, not default accuracy.
- Awareness of fairness and distributional impact, not just aggregate performance.
- A usable, clear interface that the retention team could actually try.
- A write-up that a non-data-scientist leader (e.g. a Customer Experience director) could read and act on.

What matters most

Clear reasoning, practical execution, reproducibility, and evidence that you can turn a noisy, imbalanced signal into a useful ML product for a retention team.

10. Helpful references for candidates

These resources are optional and intended to support candidates who want a starting point.

Topic	Suggested references
The NPS problem in telcos	TM Forum 'Improving Net Promoter Score using machine learning' (2021). Kannan et al. 'Prediction of Customer Transactional Net Promoter Score (tNPS) Using Machine Learning: A Telecommunication Company Case Study' (2022).

ARTEFACT

Dataset	IBM Telco Customer Churn (11.1.3+) Kaggle and IBM Cognos samples. Look for the 'Satisfaction Score' column.
Ordinal classification	mord (Python) documentation. scikit-learn OrdinalEncoder / OrdinalRegression references. CORAL / CORN frameworks for deep ordinal classification.
Imbalanced learning	imbalanced-learn documentation (SMOTE, class weighting, threshold tuning). scikit-learn metrics documentation (balanced accuracy, weighted F1, Cohen's kappa).
Gradient boosting	XGBoost, LightGBM, CatBoost documentation.
Tabular foundation models	TabPFN (PriorLabs) documentation and the TabPFN-2.5 paper (arXiv 2511.08667). TabICL paper and repository.
Interpretability	SHAP documentation. LIME documentation. scikit-learn inspection module.
Fairness auditing	Fairlearn documentation. Aequitas toolkit. scikit-learn model evaluation across groups.
LLMs for verbatim generation and classification	OpenAI / Anthropic / Mistral API documentation. sentence-transformers for embeddings. Hugging Face datasets and transformers for local models.
Interfaces	Streamlit documentation. Gradio documentation.
Monitoring and drift	Evidently AI documentation. WhyLabs / whylogs.